Part 1. Shared-Memory Programming with Threads

1.1

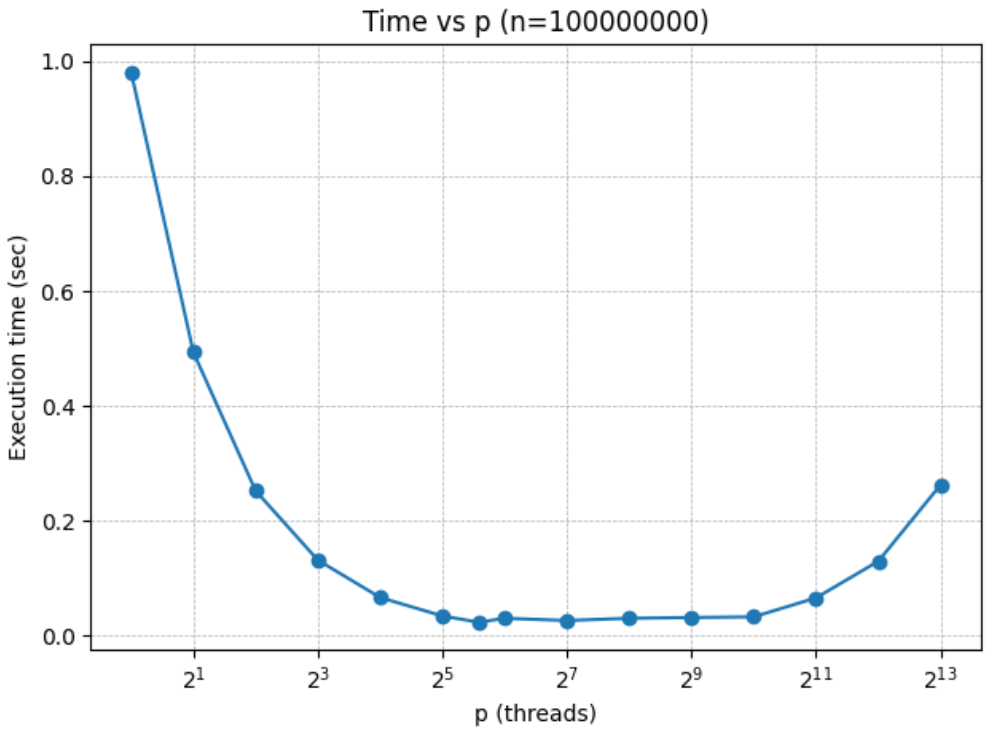

Figure 1: Execution time versus number of threads for Case 1.

1.2

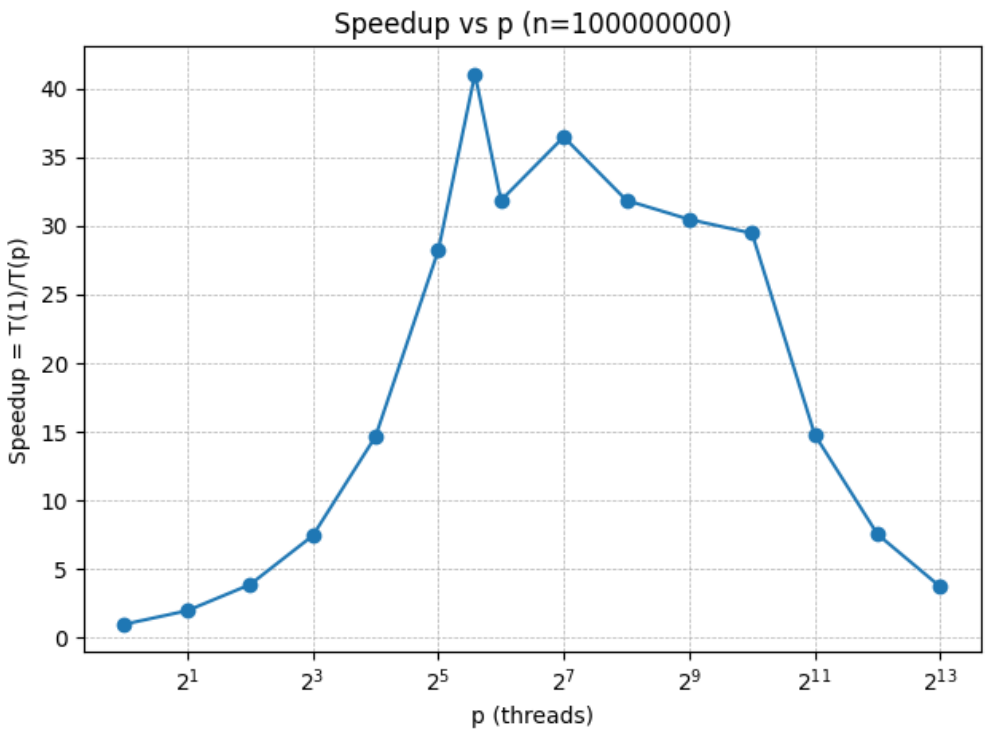

Figure 2: Execution time versus number of threads for Case 2.

1.3

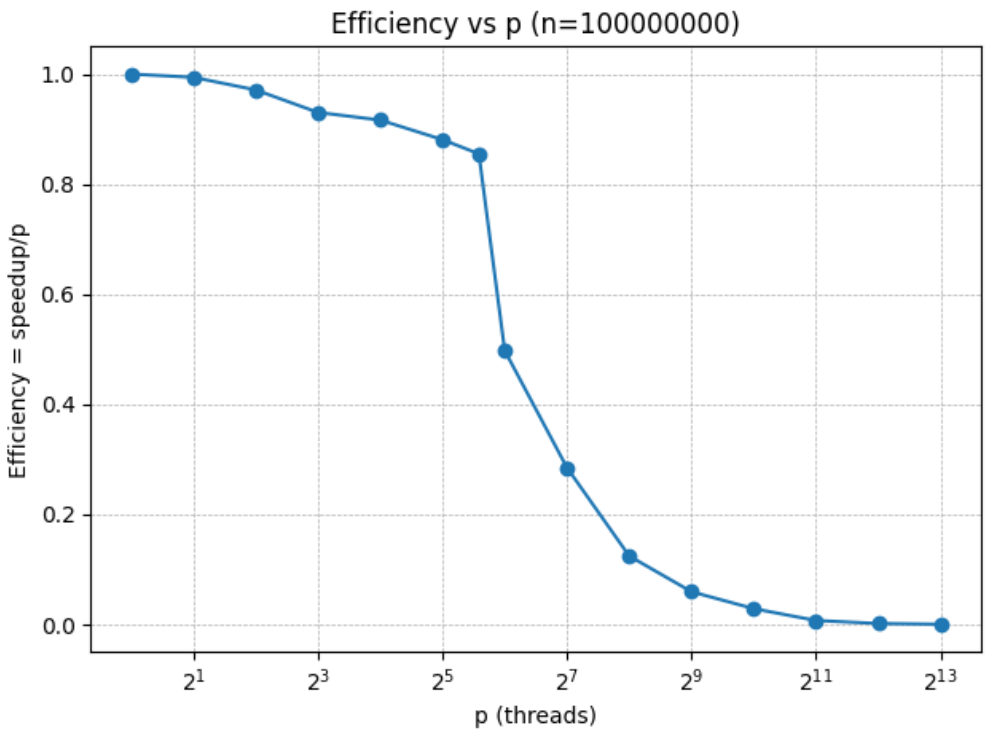

Figure 3: Execution time versus number of threads for Case 3.

1.4

Based on the experimental results, the parallel runtime is minimized when p = 128 threads, where the execution time reaches its minimum value of approximately 0.027 seconds.

2.1

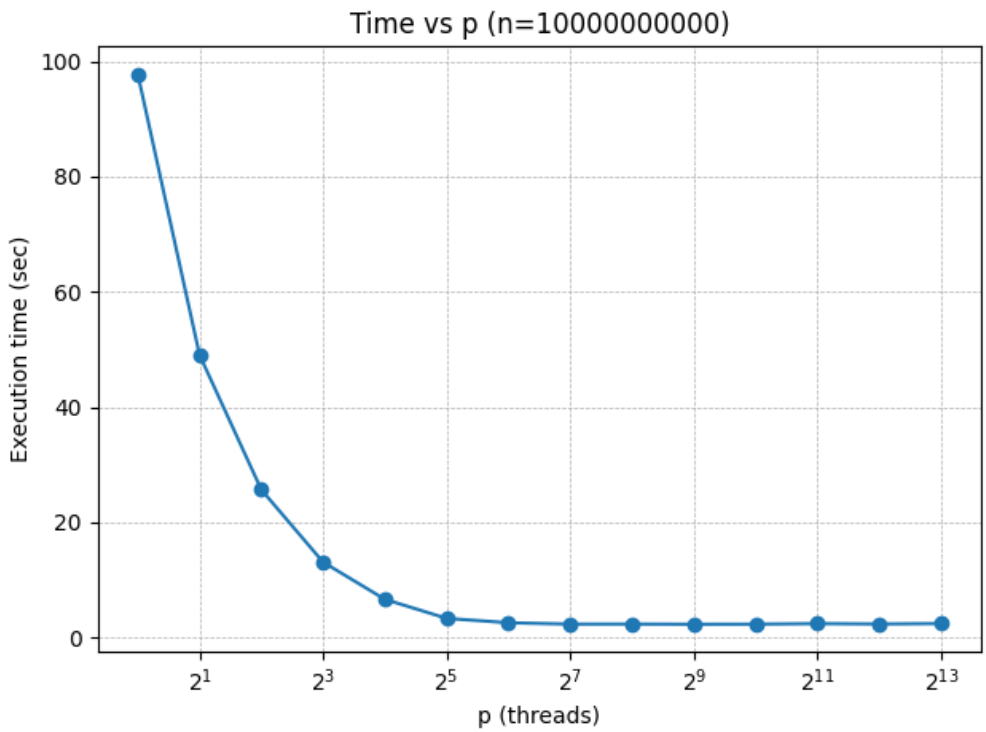

Figure 4: Execution time versus number of processes for $n = 10^{10}$.

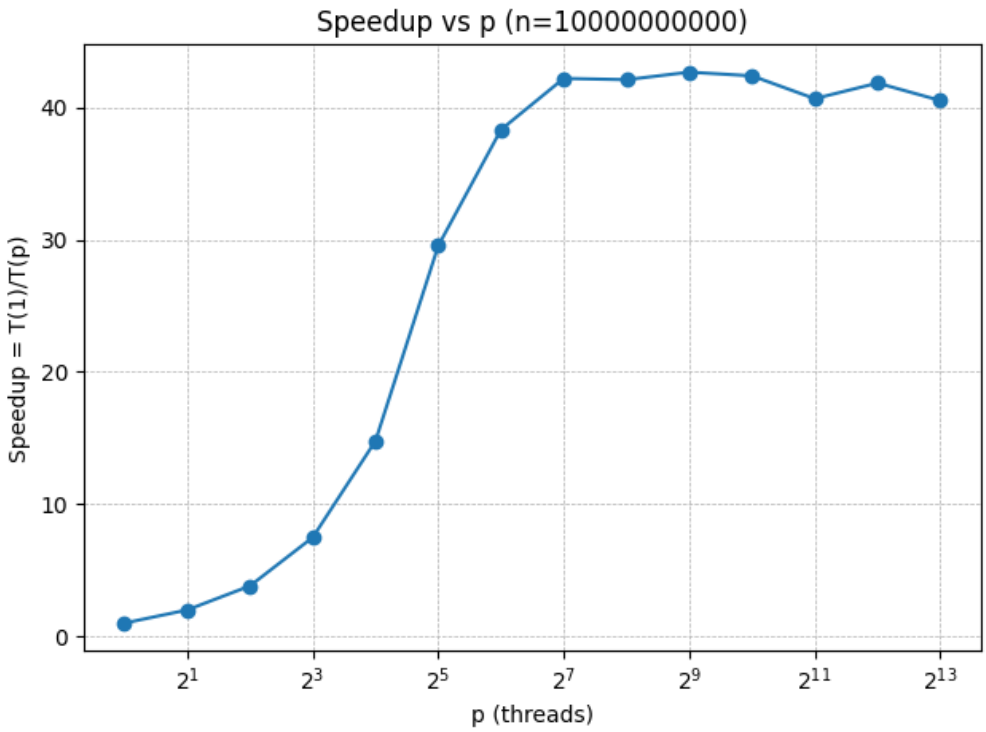

Figure 5: Speedup versus number of processes for $n = 10^{10}$.

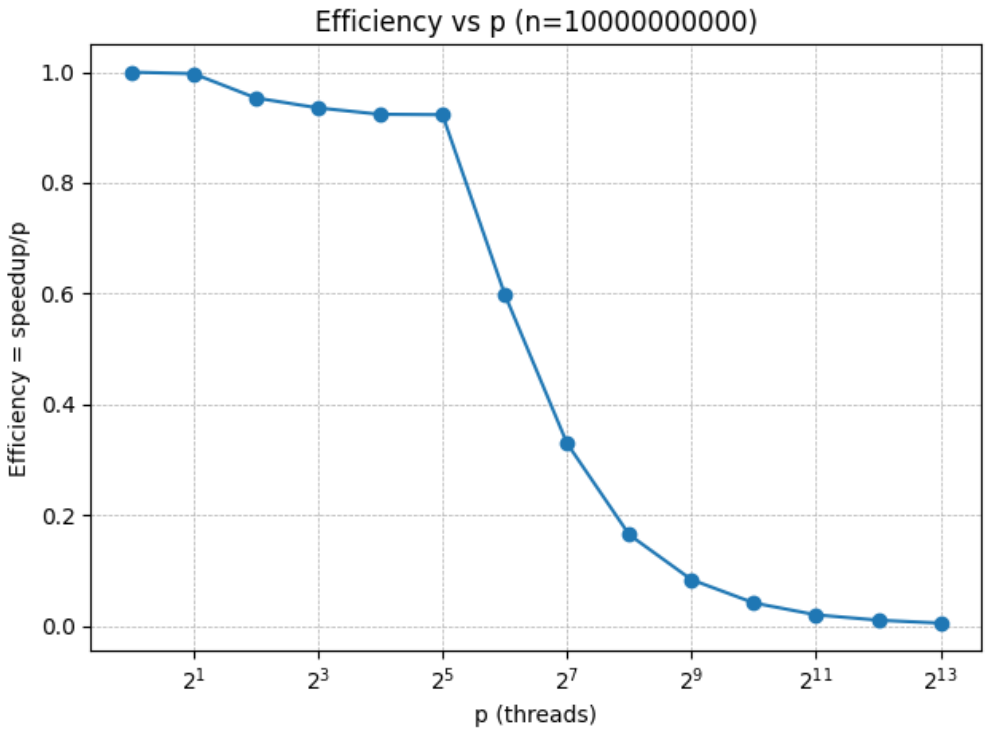

Figure 6: Parallel efficiency versus number of processes for $n = 10^{10}$.

Based on the experimental results, the parallel runtime is minimized when p = 512 threads, where the execution time reaches its minimum value of approximately 2.2912 seconds.

2.2

Yes. As p increases beyond a certain point, parallel overheads such as communication, synchronization, and process management dominate the execution time, causing the runtime to increase. This behavior is observed in our experiments, where execution time increases after the optimal value of p.

3

Yes. The optimal number of threads needed to minimize execution time depends on the problem size n. Larger values of n can better amortize parallel overhead, often requiring more threads to achieve minimum runtime. This difference is observed in our experiments, where the optimal value of p shifts as n increases.

4

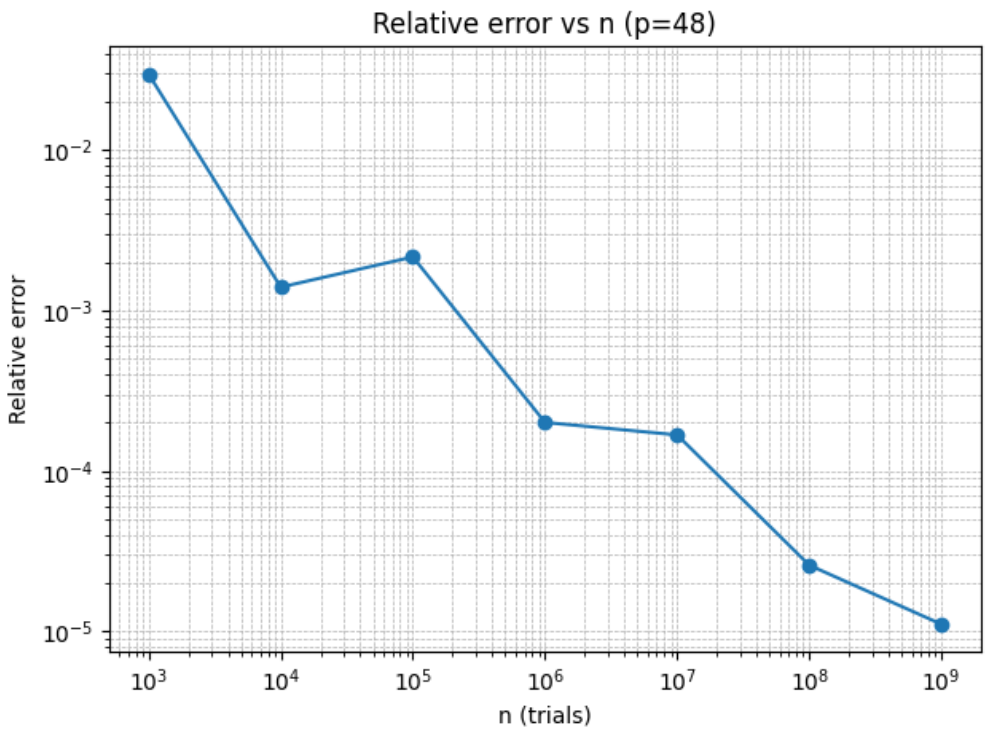

Figure 7: Error versus n illustrating the accuracy of the algorithm using $p = 48$.

Part 2. Distributed-Memory Programming with MPI

5.1

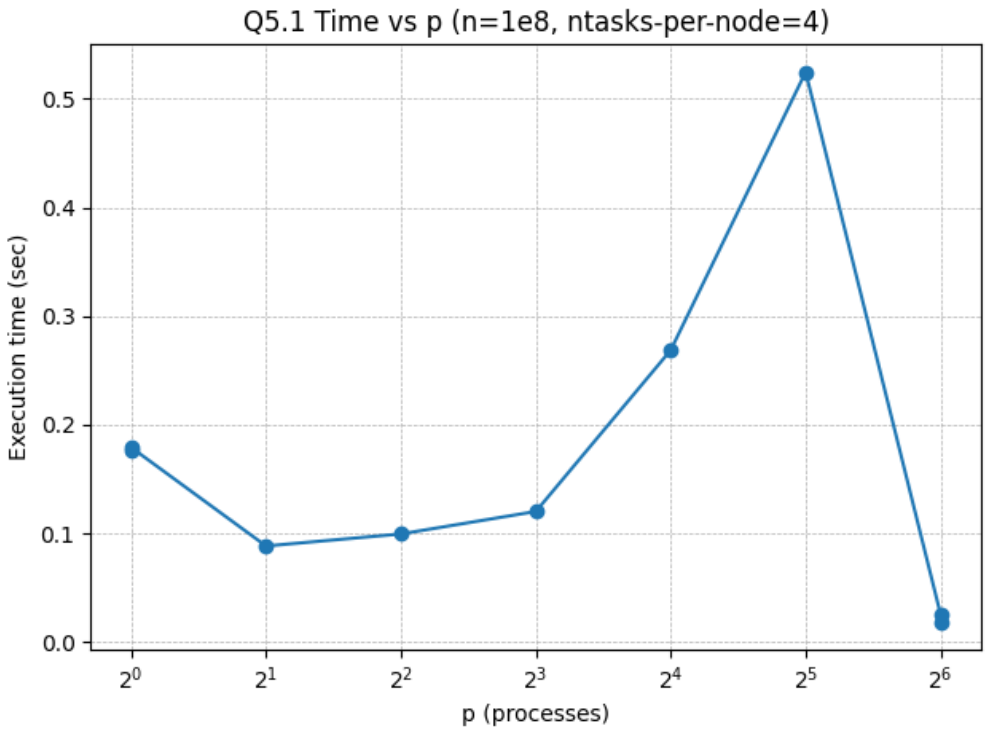

Figure 8: Execution time versus number of processes p for $n = 10^8$.

5.2

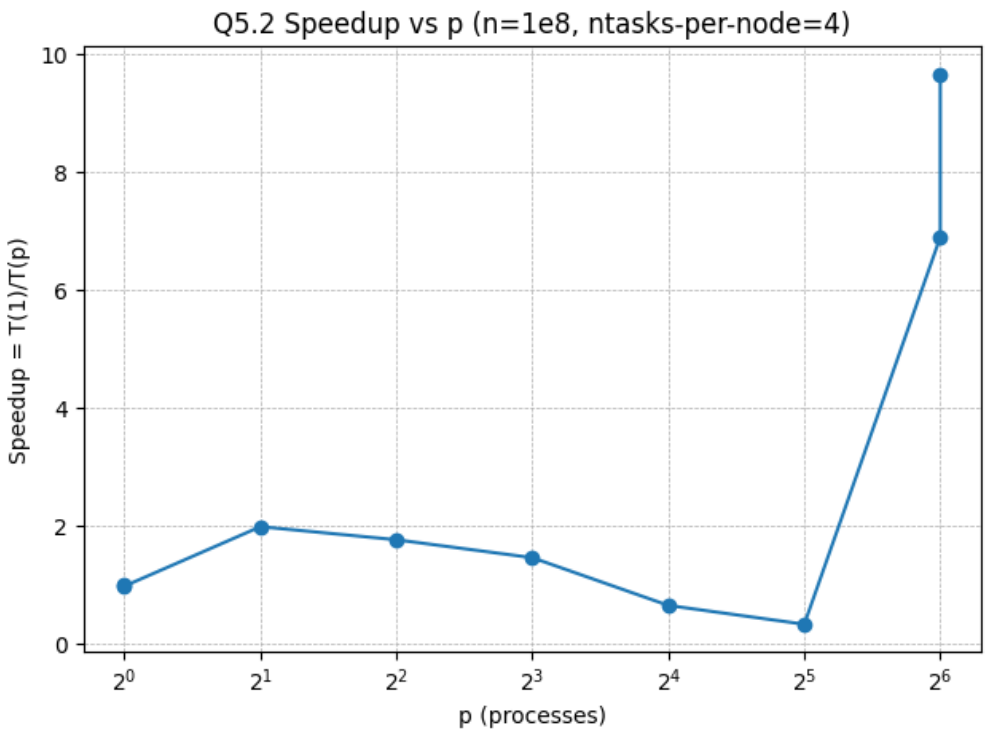

Figure 9: Speedup versus number of processes p for $n = 10^8$.

5.3

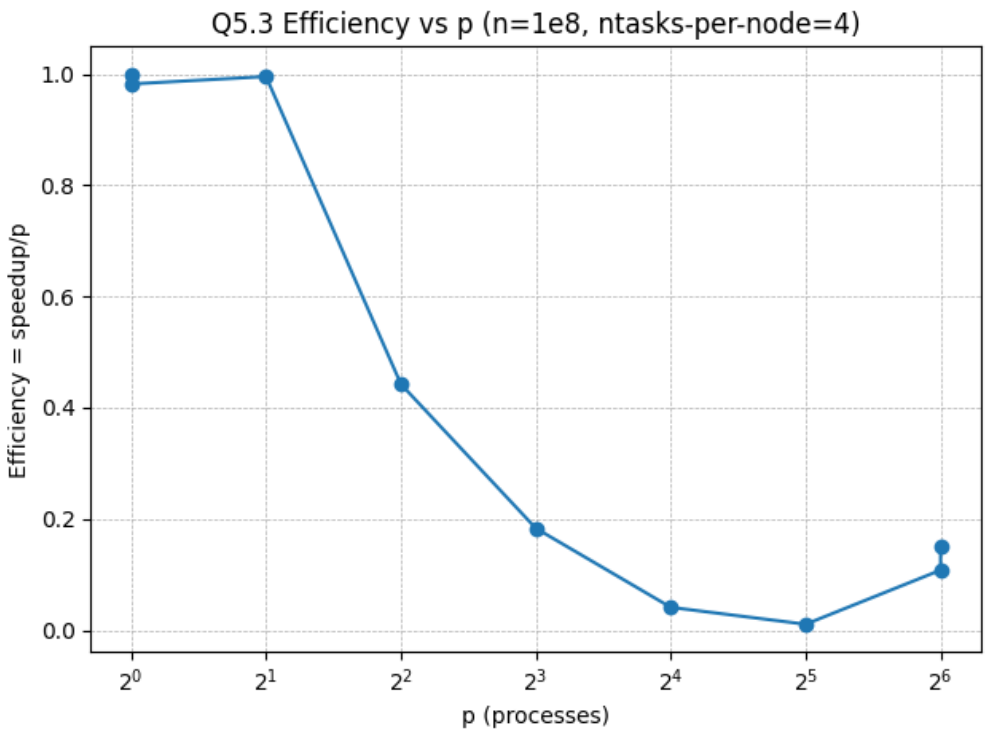

Figure 10: Parallel efficiency versus number of processes p for $n = 10^8$.

5.4

Based on the experimental results, the parallel runtime is minimized at $p = 64$, which corresponds to the lowest execution time observed in Figure 8.

6

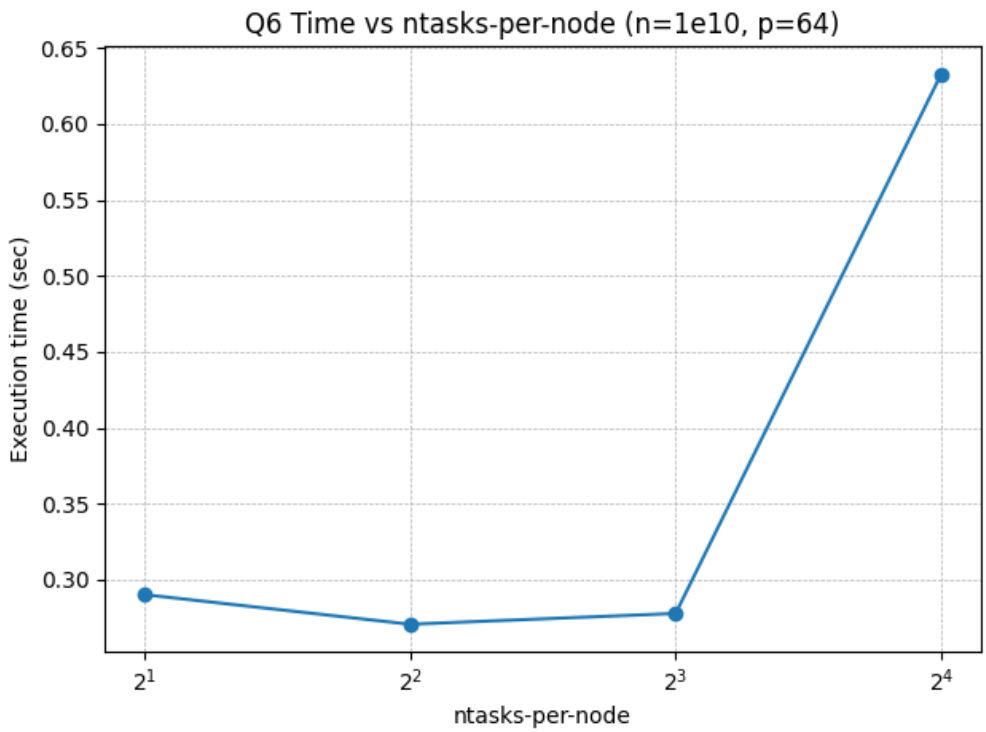

Figure 11: Execution time versus ntasks-per-node for $n = 10^{10}$ and $p = 64$.

Based on the experimental results, the total execution time is minimized at the value of `4` corresponding to the lowest point in Figure 11.

7.1

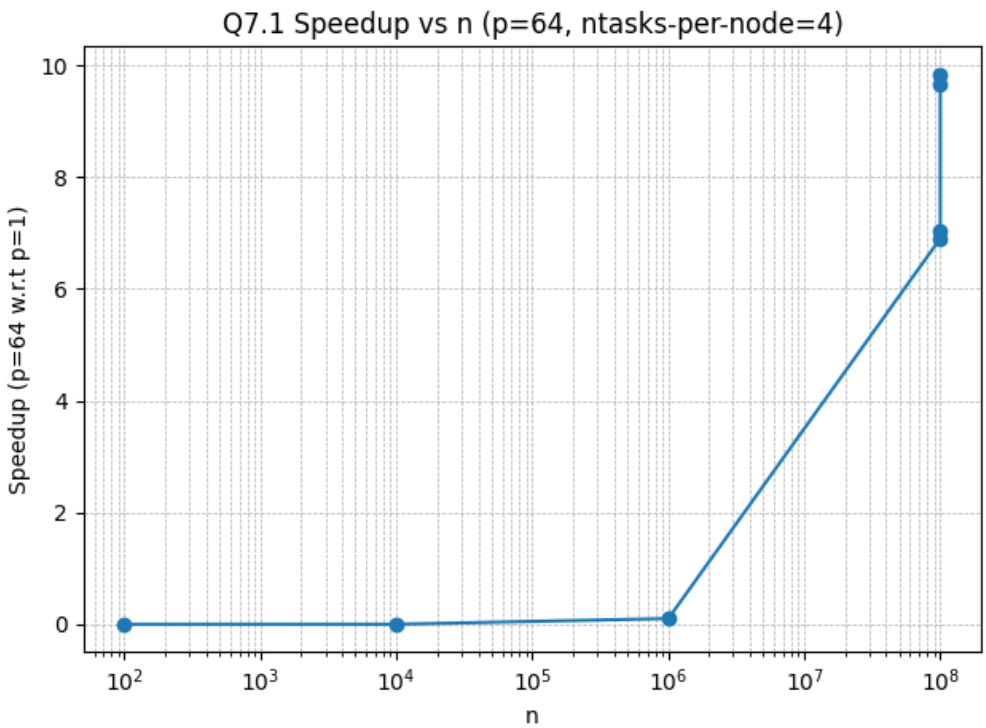

Figure 12: Speedup versus n on $p = 64$ w.r.t. $p = 1$ (ntasks-per-node=4).

7.2

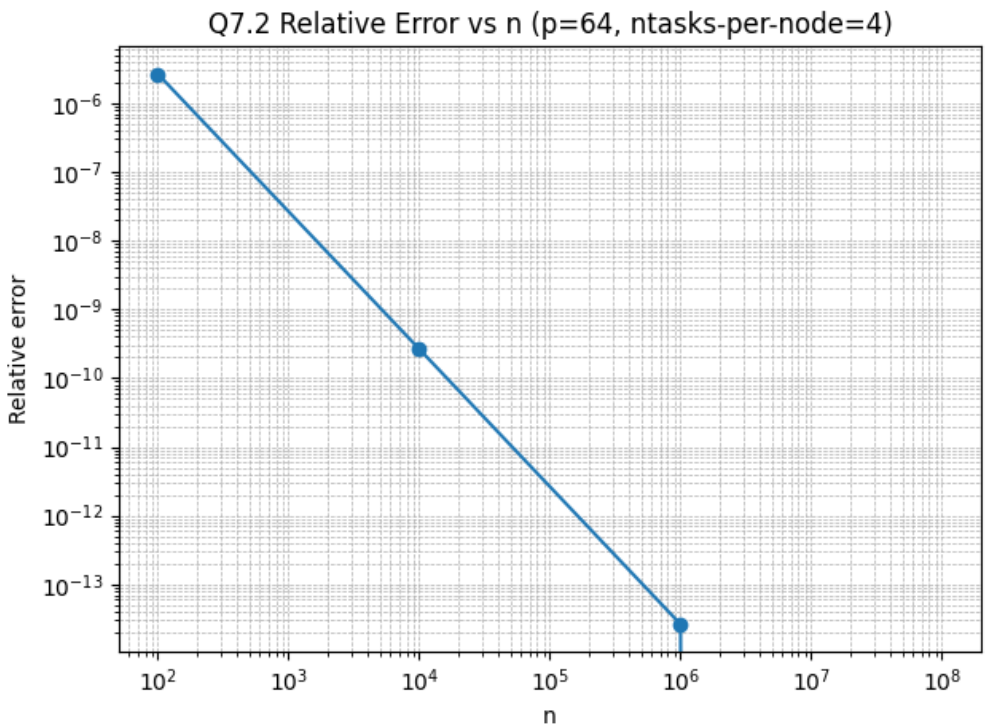

Figure 13: Relative error versus n on $p = 64$ (ntasks-per-node=4).